

Caches Inteligentes em RISC-V: Implementação em FPGA do Algoritmo de Substituição Hawkeye

Jansen Avila¹, Filipe Teixeira¹, Leonardo Borges¹

¹Engenharia de Computação – Universidade Federal do Pampa (UNIPAMPA)
Bagé – RS – Brasil

jansenavila.aluno@unipampa.edu.br

filiperosa.aluno@unipampa.edu.br

leonardoab.aluno@unipampa.edu.br

Abstract. *Cache replacement policies based on recency, such as LRU, tend to struggle whenever a workload's access pattern departs from what recency alone can predict. This work asks whether the Hawkeye algorithm – which learns from Belady's optimal solution to tell cache-friendly accesses from cache-averse ones – can do better, applied to the unified L2 cache of a two-level hierarchy for a RISC-V (RV32I) processor targeting a Cyclone III EP3C25F324C6 FPGA.*

Resumo. *Políticas de substituição baseadas em recência, como a LRU, tendem a falhar sempre que o padrão de acesso de uma carga foge do que a recência sozinha consegue prever. Este trabalho investiga se o algoritmo Hawkeye – que aprende com a solução ótima de Belady para distinguir acessos cache-friendly de cache-averse – consegue fazer melhor, aplicado à cache L2 unificada de uma hierarquia de dois níveis para um processador RISC-V (RV32I) com FPGA Cyclone III EP3C25F324C6 como alvo.*

1. Introdução

A distância crescente entre a velocidade dos processadores e a latência da memória principal — o chamado *memory wall* — torna a hierarquia de cache um dos componentes mais determinantes do desempenho de um sistema computacional. Dentro dessa hierarquia, a **política de substituição de blocos** decide, a cada falta (*miss*), qual linha da cache será sacrificada para abrigar o novo dado. A política LRU (*Least Recently Used*) e suas aproximações dominam os projetos comerciais por sua simplicidade, mas falham sistematicamente em padrões de acesso comuns, como varreduras (*streaming*) e conjuntos de trabalho maiores que a capacidade da cache, situações em que a recência deixa de ser um bom preditor de reúso [Jain and Lin 2016].

O limite teórico de qualquer política de substituição é o algoritmo ótimo de Belady (MIN), que descarta sempre o bloco cuja próxima reutilização está mais distante no futuro. Por depender de conhecimento do futuro, MIN é irrealizável diretamente em hardware. O algoritmo **Hawkeye** [Jain and Lin 2016], vencedor do 2º Campeonato de Substituição de Cache (CRC-2) [Jain and Lin 2017], contorna essa limitação com uma observação elegante: embora o futuro seja desconhecido, o *passado* é perfeitamente conhecido — é possível, portanto, simular o que Belady *teria feito* com os acessos já ocorridos e usar essas decisões como rótulos para treinar um preditor. A substituição de cache

é assim convertida em um problema de aprendizado supervisionado executável em hardware.

Este trabalho, tem por objetivo projetar, implementar, validar e avaliar o Hawkeye como política de substituição da cache L2 de um processador RISC-V (RV32I), respeitando as restrições de área e temporização do FPGA Cyclone III EP3C25F324C6. Buscando quantificar o custo da “inteligência” adicionada — em elementos lógicos, blocos de memória, frequência máxima (F_{max}) e latência de decisão — em contraste com o ganho de taxa de acerto sobre o *baseline* LRU, consolidado no Índice de Eficiência de Custo (IEC).

2. Fundamentação Teórica

2.1. O algoritmo ótimo de Belady

O algoritmo MIN de Belady minimiza o número de faltas de uma cache ao expulsar, em cada substituição, o bloco cujo próximo reuso está mais distante no futuro. Por depender de conhecimento antecipado dos acessos, MIN não é realizável diretamente em hardware, mas define o limite teórico de qualquer política prática. A ideia central do Hawkeye é reconstruir, a posteriori, essas decisões sobre acessos já ocorridos, usando-as como sinal de treinamento para uma política preditiva.

2.2. OPTgen

O OPTgen reconstrói essas decisões retroativamente. Para conjuntos amostrados, mantém um *vetor de ocupação* que registra quantas linhas estariam “vivas” a cada instante. Ao re-acesar um bloco, define-se o *intervalo de uso* entre esse acesso e o anterior: se o número de linhas vivas nesse intervalo sempre foi inferior à associatividade, havia espaço para manter o bloco, e o acesso é rotulado *hit* do OPT; caso contrário, *miss*. Por contabilizar linhas vivas, e não blocos distintos, o OPTgen pode declarar *hit* mesmo onde uma política de recência, como a LRU, teria expulsado o bloco.

2.3. Sampler

Como manter esse vetor para todos os conjuntos seria proibitivo, o Hawkeye restringe o OPTgen a um subconjunto amostrado, monitorado pelo *sampler* — uma cache de metadados que registra o histórico de acessos, a assinatura de reuso (tipicamente derivada do PC) e o instante de cada bloco observado, delimitando o intervalo de uso sem replicar a cache completa para fins de simulação.

2.4. Preditor Hawkeye

Os rótulos do OPTgen treinam o preditor, uma tabela de contadores saturantes indexada por *hash* do PC: se o OPT mantinha o bloco, o contador associado é incrementado; caso contrário, decrementado. A premissa é que o comportamento de reuso se correlaciona com a instrução de origem — um mesmo *load* dentro de um laço tende a ser consistentemente amigável ou hostil à cache. Na inferência, o bit mais significativo do contador classifica o acesso como *cache-friendly* ou *cache-averse*, os dois lados do limiar de um único contador saturante.

2.5. RRIP

A predição converte-se em ação de substituição pelo arcabouço RRIP (*Re-Reference Interval Prediction*): cada linha carrega um valor RRPV que estima a distância até o próximo reuso — blocos *friendly* entram com RRPV baixo, *averse* com RRPV alto, acertos zeram o RRPV da via, e a vítima é a primeira via com RRPV máximo (ou o conjunto inteiro envelhece até que uma surja). O SHiP explora ideia semelhante, mas sem o rótulo derivado da solução ótima que caracteriza o Hawkeye.

3. Arquitetura Proposta

3.1. Visão geral da hierarquia de memória

A hierarquia implementada é composta por duas caches L1 privadas — uma de instruções (L1I) e uma de dados (L1D) — e uma cache L2 unificada, na qual o Hawkeye atua como política de substituição. A Tabela 1 resume os parâmetros de cada nível, incluindo a decomposição do endereço de 32 bits em *tag*, índice e *offset*.

Tabela 1. Configuração da hierarquia de memória.

Parâmetro	Cache L1 (I/D)	Cache L2 (unificada)
Capacidade	4 KB	32 KB
Associatividade	2 vias	8 vias
Tamanho do bloco	32 bytes	64 bytes
Política de substituição	LRU	Hawkeye
<i>Tag</i>	21 bits	20 bits
Índice	6 bits	6 bits
<i>Offset</i>	5 bits	6 bits

Restringir o Hawkeye à L2, mantendo LRU nas L1, é uma escolha deliberada, fundamentada empiricamente na fase de modelagem (Seção 5): em caches pequenas e de baixa associatividade o custo da predição supera o benefício, e a L1 exige latência de acerto mínima. A L2, por concentrar os acessos que sobreviveram ao filtro da L1 — justamente os padrões em que a LRU mais falha —, é o ponto de maior retorno para a política preditiva.

3.2. Fluxo de dados da L2

O caminho de leitura da L2 é integralmente combinacional entre dois elementos sequenciais: a partir do endereço registrado, operam em cascata os comparadores de *tag* de cada via (em paralelo), uma árvore OU para detecção de acerto, um codificador da via vencedora e os estágios de multiplexação do dado. Um acerto custa de 1 a 3 ciclos de relógio, conforme o estagiamento do *pipeline*; uma falta é governada por uma máquina de estados finitos que orquestra a seleção de vítima, o *write-back* quando necessário e o preenchimento a partir da memória principal.

O PC da instrução causadora do acesso é transportado explicitamente por campos adicionais dos registradores de *pipeline* até o estágio de memória, onde alimenta o preditor. Como otimização, o *hash* do PC é computado precocemente, de modo que apenas ~11 bits (o índice do preditor) precisam ser carregados adiante, em vez dos 32 bits originais.

3.3. O subsistema Hawkeye em hardware

O subsistema Hawkeye instancia e interliga em hardware os componentes conceituais descritos na Seção 2. Um módulo de *hash*, de lógica puramente combinacional, aplica um CRC-32 (polinômio $0 \times \text{EDB88320}$) sobre o endereço e o PC de entrada, produzindo o índice do preditor (11 bits, para 2048 entradas) e a assinatura de 8 bits consumida pelo sampler. O sampler é implementado como uma cache amostral de 64 conjuntos por 8 entradas, cuja busca por assinatura é resolvida por um banco de comparadores paralelos e cuja substituição, por um seletor de vítima dedicado. O OPTgen mantém seu vetor de ocupação em uma estrutura de 128 posições, atualizada a cada consulta delimitada pelos *timestamps* fornecidos pelo sampler. O preditor é uma tabela de 2048 contadores saturantes parametrizáveis (3 bits, conforme o modelo de referência [Jain and Lin 2016]), cujo bloco elementar suporta incremento, decremento e envelhecimento por deslocamento, com saída de inferência combinacional.

A coordenação de todo esse fluxo cabe a um controlador de treinamento, implementado como uma máquina de estados finitos de nove estados que serializa o processo a cada acesso válido: havendo acerto no sampler, o OPTgen é consultado, o PC previamente armazenado é treinado conforme o veredito, e então o vetor de ocupação e a entrada do sampler são atualizados. Esse treinamento ocorre inteiramente fora do caminho crítico de acerto da cache, pois a inferência — a leitura do contador — permanece combinacional.

3.4. Integração com a política de substituição

Um módulo de política conecta o subsistema Hawkeye à interface da L2. Ele mantém a tabela de valores RRPV (3 bits por via) e implementa as regras de inserção e a ordem de prioridade na seleção de vítima definidas pelo arcabouço RRIP, resumidas na Tabela 2: se nenhuma via satisfaz as duas primeiras condições de vítima, o conjunto inteiro envelhece ($\text{RRPV} += 1$ em todas as vias) até que uma via atinja RRPV máximo. O controle do sistema completo é composto por quatro máquinas de estados finitos, responsáveis, respectivamente, pela hierarquia de memória como um todo, pela cache L2, pelo controlador de treinamento do Hawkeye e pelo OPTgen.

Tabela 2. Valores de RRPV por evento e ordem de prioridade na seleção de vítima, conforme a política RRIP.

Evento / Critério de vítima	RRPV	Prioridade
Acerto (<i>hit</i>)	0	Retenção máxima
Preenchimento <i>friendly</i>	2	Retenção alta
Preenchimento <i>averse</i>	6	Retenção baixa
Envelhecimento (<i>aging</i>)	+1 até 7	Retenção decrescente
Via inválida (nunca preenchida)	—	1ª vítima
Primeira via com RRPV = 7	—	2ª vítima
Nenhuma das anteriores	—	envelhece o conjunto

3.5. Mapeamento de software para hardware

A implementação seguiu duas etapas complementares: primeiro, um modelo de referência em alto nível, escrito em linguagem C, foi usado para validar a lógica do algoritmo e

executar os *benchmarks* de comparação contra o *baseline* LRU; em seguida, essa lógica foi levada aos desenhos RTL, com implementação e validação em Verilog buscando a integração com a arquitetura da hierarquia de cache propriamente dita.

4. Metodologia

4.1. Planejamento e gestão do projeto

O desenvolvimento seguiu um processo iterativo, com entregas periódicas e gestão visual apoiada em um quadro Kanban, o que permitiu acompanhar o progresso do trabalho e organizar as tarefas por responsável e critério de conclusão. O trabalho teve início no estudo do artigo original do Hawkeye [Jain and Lin 2016] e dos conceitos de suporte que embasam a Seção 2, a partir do qual foram definidos os requisitos funcionais do modelo de referência.

4.2. Modelo de referência em software

A hierarquia de memória foi primeiro materializada em um modelo de referência em linguagem C, com a política de substituição selecionável em tempo de execução entre LRU e Hawkeye, de forma independente em cada nível da cache. Esse mecanismo permitiu comparar diretamente as três configurações de interesse — LRU+LRU, HE+HE e LRU+HE — sob *traces* idênticos, executando em alto nível a mesma lógica de treinamento descrita na Seção 3.3.

4.3. Projeto RTL e verificação

Em paralelo à consolidação dos *benchmarks*, iniciou-se a transição para hardware, com os desenhos RTL dos cinco componentes principais (cache L1, cache L2, OPTgen, *sampler* e preditor) e uma síntese exploratória com a ferramenta de código aberto Yosys, para detecção precoce de construções não sintetizáveis. Todos os módulos foram então descritos em Verilog e verificados com *testbenches* no Icarus Verilog e no ModelSim.

4.4. Síntese e otimização

O projeto foi submetido ao fluxo completo do Quartus II 13.1 (Análise & Síntese, *Fitter* e temporização via TimeQuest), com restrição de 50 MHz e o EP3C25F324C6 como dispositivo-alvo. A etapa final concentrou-se na extração de métricas, índices de mérito e análise da arquitetura.

5. Resultados de Síntese e Desempenho

5.1. Validação funcional e desempenho em software

A avaliação em software ocorreu em duas rodadas, e o contraste entre elas é, em si, um resultado. Na primeira rodada, conduzida com cargas de acesso convencionais, LRU e Hawkeye produziram taxas de acerto praticamente indistinguíveis na L2 (Tabela 3). Esse empate não foi um defeito do algoritmo, mas um sintoma de que as cargas não impunham pressão de conflito suficiente para diferenciar as políticas e de que a falta de um PC representativo impedia o preditor de distinguir instruções *friendly* de *averse*.

Diagnosticadas essas limitações, os *benchmarks* foram reprojatados para exercer pressão adversarial genuína e receber PCs significativos. A Tabela 4 apresenta o resultado dessa segunda rodada, agora comparando as três configurações de política sobre

Tabela 3. Primeira rodada — taxa de acerto na L2 (HR-L2) com cargas convencionais (L1 4 KB/2 vias, L2 32 KB/8 vias).

Benchmark	LRU	Hawkeye	Δ p.p.
Convolução 256×256	$\approx 50,00\%$	$\approx 50,00\%$	≈ 0
Linked List	$\approx 50,00\%$	$\approx 50,00\%$	≈ 0
Pattern	$\approx 50,00\%$	$\approx 50,00\%$	≈ 0
Streaming	50,40%	52,20%	+1,8

Tabela 4. Segunda rodada — taxa de acerto na L2 (HR-L2) por configuração de política em cargas adversariais, e variação de ciclos da configuração adotada (LRU+HE) em relação ao *baseline* LRU+LRU (L1 4 KB/2 vias, L2 32 KB/8 vias).

Benchmark	LRU+LRU (HR-L2)	HE+HE (HR-L2)	LRU+HE (adotado)	Δ Ciclos (LRU+HE)
Streaming + Hot	44,44%	48,67%	48,67%	−350k
Conv 256×256	50,00%	66,46%	49,91%	≈ 0
Linked List	14,97%	14,78%	16,06%	−98k
Pattern Search	1,83%	25,97%	26,02%	−1.124k

traces idênticos. Com cargas cujos conjuntos de trabalho excedem a associatividade, as diferenças tornam-se expressivas.

O caso mais expressivo é o *Pattern Search*, saltando de *thrashing* completo sob LRU para uma cache funcional sob Hawkeye. A Convolução ilustra um resultado paradoxal que justifica a escolha arquitetural do projeto: sob a configuração HE+HE, a taxa de acerto na L2 melhora, mas o total de ciclos piora, porque o Hawkeye aplicado à L1 de apenas 2 vias degrada esse nível sensível à latência e gera faltas adicionais. A configuração adotada, LRU+HE, evita esse efeito: mantém a L1 em LRU e obtém na L2 um resultado essencialmente equivalente ao *baseline* nessa carga amigável, sem penalidade de ciclos. Em conjunto, os dados mostram que LRU+HE captura os ganhos nos casos adversariais sem incorrer no prejuízo que o Hawkeye causaria na L1.

5.2. Resultados de síntese

Esta seção reúne as métricas de área, temporização, latência de decisão e o Índice de Eficiência de Custo (IEC), consolidadas após a re-síntese final. A Tabela 5 compara o projeto sintetizado com o subsistema Hawkeye ativo contra um *baseline* sintetizado sob a restrição de 50 MHz no *corner* Slow 1200mV 85°C (o pior caso entre os três *corners* analisados).

O subsistema Hawkeye (*sampler*, OPTgen e preditor) responde sozinho por 71,3% do total de LEs do projeto. O IEC, calculado como o ganho percentual de taxa de acerto dividido pelos LEs extras e usando o resultado mais expressivo do *Pattern Search* (+24,19 pontos percentuais sobre 11.843 LEs adicionais), resulta em aproximadamente 0,00204 pontos percentuais de ganho por LE, ou cerca de 2,04 pontos percentuais a cada mil LEs adicionados.

Tabela 5. Área e temporização — *baseline* sem política preditiva vs. projeto com Hawkeye ativo na L2.

Métrica	Baseline	Com Hawkeye	Impacto
Elementos lógicos (LEs)	4.513	16.356	+262,4%
Registradores dedicados	3.168	6.542	+106,5%
Bits de memória embarcada	10.752	55.296	+414,3%
F_{max} (MHz)	89,23	62,81	−29,6%
<i>Slack</i> de <i>setup</i> (ns)	+8,793	+4,079	—
TNS (ns)	0,0	0,0	—

A Tabela 6 decompõe a latência de decisão medida em hardware. O caminho de acerto interno da L2 é puramente combinacional (1 a 3 ciclos, conforme o estágio do *pipeline*, já descrito na Seção 3); os valores abaixo medem a latência de ponta a ponta observada pela CPU, da requisição até a resposta completa, incluindo a verificação prévia da L1 e a máquina de estados de roteamento da hierarquia.

Tabela 6. Latência de decisão medida em hardware (ciclos de *clock*).

Evento	Ciclos
Acesso com <i>hit</i> na L2 (fim a fim)	10
Acesso com <i>miss</i> e substituição (fim a fim)	15
Seleção de vítima RRIP	10
Treinamento do preditor Hawkeye	17

O treinamento do preditor conclui-se 17 ciclos após o início do acesso — depois, portanto, da resposta ao processador já ter sido entregue em 15 ciclos — confirmando experimentalmente que o treinamento de fato continua em segundo plano, fora do caminho crítico de resposta.

Tabela 7. Tabela de referência para comparação (Apêndice B da especificação) — Benchmark *Pattern Search* (L1 4 KB/2 vias, L2 32 KB/8 vias).

Métrica	Baseline (LRU)	Hawkeye	Impacto
Taxa de Acerto L2 (%)	1,83%	26,02%	+24,19 p.p.
Área (LEs)	4.513	16.356	+262,4%
Frequência (F_{max})	89,23 MHz	62,81 MHz	−29,6%
Latência de Decisão	1 ciclo	15 ciclos	+14 ciclos

6. Análise Crítica

A pergunta central que orienta esta análise é se o ganho de taxa de acerto obtido justifica o custo em silício adicionado pela política. A resposta, à luz dos dados apresentados, é condicional, e as próprias condições que a delimitam são instrutivas.

Os ganhos do Hawkeye concentram-se exatamente nos regimes em que a LRU colapsa, isto é, em conjuntos de trabalho maiores que a associatividade e em padrões de

conflito sistemático. No *Pattern Search*, o salto de 1,83% para 26,02% de acerto na L2 não representa um refinamento marginal, mas a diferença entre uma cache praticamente inútil e uma cache funcional, com reflexo direto de cerca de 1,12 milhão de ciclos economizados. Em cargas já amigáveis à LRU, como a Convolução com *blocking* adequado, o Hawkeye apenas empata, pois o preditor converge para o mesmo comportamento e o custo adicional não retorna benefício — reforçando que a política preditiva é mais valiosa como um seguro contra o pior caso do que como aceleradora do caso médio.

Cabe registrar uma limitação metodológica do modelo de software: sem o PC real emitido por um fluxo de instruções, o preditor opera com assinaturas atribuídas artificialmente às cargas, subestimando sua capacidade de distinguir instruções *friendly* de *averse*. A integração do RTL ao núcleo RV32I, que fornece o PC autêntico ao estágio de memória, tende a favorecer o Hawkeye em relação ao que o modelo de referência indica.

A validação em RTL, reproduzindo os mesmos quatro padrões de acesso diretamente em hardware sobre a hierarquia completa (Tabelas 8), confirma esse comportamento nos regimes de folga (*Streaming*, Convolução) e expõe o pior caso absoluto de conflito de associatividade no *Pattern Search* e no percurso de lista encadeada — mais severo que o observado no modelo em software, por não contarem com nenhuma folga ocasional de localidade nesses dois padrões. A Figura 1 ilustra esse contraste.

Tabela 8. Validação em hardware — HR-L1, HR-L2 e custo em ciclos (modelo L1= 1, L2= 10, RAM= 100 ciclos por acesso) por padrão de acesso, mesma quantidade de requisições para os quatro padrões.

Benchmark	HR-L1 (%)	HR-L2 (%)	Ciclos
Streaming + Hot Conflitante	85,94	55,11	397.418
Convolução 256×256	95,67	50,00	166.856
Lista encadeada	0,01	0,04	4.997.813
Busca com conflito de via	0,00	0,00	5.000.000

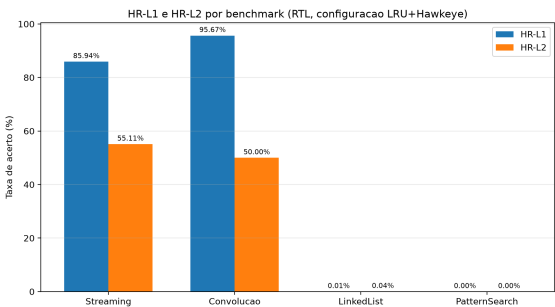


Figura 1. HR-L1 e HR-L2 por benchmark, medidos em RTL sobre a hierarquia completa com o Hawkeye ativo na L2.

Além dos quatro padrões adversariais, *testbenches* dedicados validaram a estabilidade do sistema sob cargas crescentes, de 1 mil a 500 mil requisições, forçando sistematicamente *misses* na L1 para isolar o comportamento da L2: a taxa de acerto converge rapidamente e se mantém estável a partir de poucas dezenas de milhares de acessos, sem degradação em cargas mais longas.

A evolução das classificações do preditor ao longo do mesmo intervalo de carga (Tabela 9) mostra que ele parte do estado de *reset* — 100% *friendly*, o valor padrão antes de qualquer treinamento — e converge para uma divisão estável de aproximadamente 53%/47% entre *friendly* e *averse* a partir de 60 mil requisições. Isso confirma que o Hawkeye exige uma janela de aquecimento (*warm-up*) considerável antes de expressar plenamente sua capacidade preditiva — um fator relevante para cargas de trabalho de curta duração, nas quais a política ainda estaria ”aprendendo”quando a execução termina.

Tabela 9. Evolução das classificações *friendly/averse* e do custo em ciclos em função da quantidade de requisições processadas (amostra representativa da curva de convergência).

Requisições	<i>Friendly</i> (%)	<i>Averse</i> (%)	Ciclos
2.029	100,00	0,00	112.900
20.029	60,38	39,62	1.102.900
100.029	53,13	46,87	5.502.900
600.029	53,42	46,58	33.002.900
1.000.029	55,97	44,03	55.002.900

Por fim, a validação da hierarquia completa (Figura 2) confirma que L1 e L2 estabilizam suas taxas de acerto rapidamente (por volta de 58,3% e 60,0%, respectivamente) e permanecem estáveis até 500 mil requisições, sem degradação sob operação prolongada.

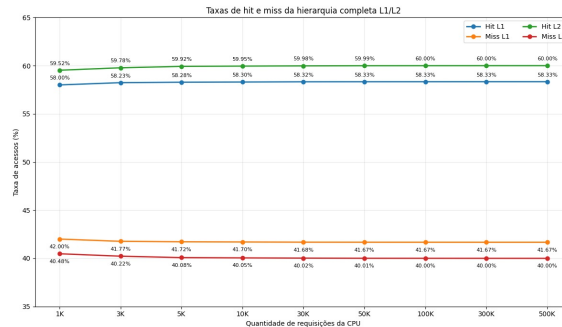


Figura 2. Taxas de *hit* e *miss* da hierarquia completa (L1 e L2) em função da quantidade de requisições da CPU.

Quanto à natureza do custo, o *overhead* do Hawkeye — concentrado no *sampler*, no OPTgen e no preditor — é essencialmente independente da capacidade da cache, o que favorece sua amortização em caches maiores; numa L2 de apenas 32 KB, esse custo fixo pesa proporcionalmente mais do que pesaria num *last-level cache* de vários megabytes, comprimindo o IEC alcançável neste projeto.

No que se refere à temporização, o projeto favorece deliberadamente o caminho de acerto: a inferência resume-se a uma leitura combinacional de contador, e todo o treinamento ocorre em segundo plano, fora do caminho crítico. Os números finais confirmam esse favorecimento: no *corner* de pior caso (Slow 1200mV, 85°C), o *slack* de *setup* é de +4,079 ns, com TNS nulo, e o F_{max} atinge 62,81 MHz — folgadoamente acima dos 50 MHz exigidos pelo SDC.

A dissipação de potência estimada segue o padrão observado na área: o custo concentra-se quase inteiramente na parcela dinâmica (Tabela 10), coerente com a natureza

combinacional e constantemente ativa da lógica adicionada pelo Hawkeye, em contraste com a parcela estática, que varia pouco entre os dois projetos.

Tabela 10. Potência dissipada — baseline sem política preditiva vs. projeto com Hawkeye ativo na L2.

Métrica	Baseline	Com Hawkeye	Impacto
Potência total	172,92 mW	249,46 mW	+44,3%
Dinâmica (núcleo)	38,21 mW	112,87 mW	+195,4%
Estática (núcleo)	81,64 mW	83,05 mW	+1,7%
E/S (I/O)	53,07 mW	53,54 mW	+0,9%

Em balanço, a LRU permanece imbatível em custo para cargas previsíveis e amigáveis. Já para padrões adversariais — percurso de estruturas encadeadas, tabelas de consulta sob pressão, *streaming* com regiões quentes — o Hawkeye oferece uma robustez que a LRU não possui, ao preço de um acréscimo de 262,4% na área e de uma redução de 29,6% no F_{max} . O IEC consolidado na Seção 5.2 quantifica esse compromisso.

7. Conclusão

O desenvolvimento deste trabalho permitiu aplicar, na prática, conceitos de arquitetura de cache que até então eram vistos apenas de forma teórica na disciplina. Implementar o Hawkeye desde o modelo em C até a síntese em Verilog trouxe uma compreensão mais concreta de como uma política de substituição realmente se comporta, e a comparação com o LRU deixou evidente que a vantagem do Hawkeye aparece principalmente em cargas de acesso mais adversas, quando a simples recência não é suficiente para prever bons candidatos à substituição.

Referências

- Jain, A. and Lin, C. (2016). Back to the future: Leveraging belady’s algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pages 78–89, Seoul, South Korea. IEEE.
- Jain, A. and Lin, C. (2017). Hawkeye: Leveraging belady’s algorithm for improved cache replacement. In *The 2nd Cache Replacement Championship (CRC-2), co-located with ISCA*, Toronto, Canada.